SubSplit

Escrow-Protected Subscription Sharing

A written companion to the project presentation

DOTE 6751 Database Systems · Group Project · November 2026

Xuanmo AN · Yi HOU · Zhao Han HUANG · Jinyu LI · Yitong NI · Shujing YE · Shiyao ZHANG

1. The problem

SubSplit addresses a specific coordination failure in the consumer subscription market. Streaming and software vendors price their family plans for a household: six seats for roughly the cost of one and a half individual subscriptions. The discount is real, but it presupposes a unit that many subscribers do not have. A student living alone, a professional who shares neither an address nor a payment card with anyone, cannot use a product that was designed for a family of six.

The substitute that has emerged is the informal group buy. Strangers assemble on social media, one of them holds the account, and the others transfer a year of fees in advance. This arrangement functions, but it carries no institution behind it. The account holder retains unilateral control of the credential and can change it at any moment; the members have paid for a period they cannot reclaim; and neither side holds a record that would survive a disagreement. The failure is not one of price but of enforcement.

SubSplit therefore does not attempt to divide a bill more efficiently. It supplies the missing institution — custody of funds and a verifiable record of conduct — so that five people who have never met can transact with the same confidence as a household. The design question behind the database is consequently narrow and concrete: what must be recorded so that a stranger's promise becomes enforceable?

2. How the platform works

The business model has four movements. A host who already owns a family plan submits the official invoice for verification and opens a group, setting a monthly price per seat. Members join until the product's seat ceiling is reached, and each of them prepays twelve months. That prepayment does not reach the host: it is held in an escrow pool. Every month the platform releases one twelfth of the prepayment to the host and retains a five per cent fee on that release. At the end of the year the escrow is empty and the host has been paid in full, in twelve instalments rather than one.

The arrangement matters because it distributes risk over time. A member who has paid for a year is not exposed to a single decision by the host: if the host changes the password in month four, the

member still holds eight months of unreleased money, and the platform can freeze the remaining balance before it moves. Conversely, a host is not asked to trust six strangers for a year; the platform pays them monthly from funds it already holds. Neither party needs the other's goodwill, which is precisely the point.

Two design choices follow directly from this model and shape the schema. First, a complaint is not an automatic refund. Detection can be automated, but judgement cannot: two coordinated members could otherwise drain any host. The platform therefore records the facts, freezes the money, and routes the case to an administrator, whose decision is itself recorded. Second, every event that costs someone money must leave a trace — a credential change, a released month, a credit penalty — because a number without a history cannot be defended in an argument.

3. The data model

The model follows the entity-relationship method and translates it into eleven relations, using the mapping rules of the course textbook. The conceptual design begins with four central entity sets: the user, the digital service, the group, and the seat. Around them sit the artefacts the business cannot do without — the credential, the escrow, the monthly ledger, the complaint, the password history, the payment instruments, and the credit record.

Three of these deserve comment because they are where the model earns its keep. The credential vault is a weak entity: a passphrase without a group is meaningless, so it borrows the group's key as its own primary key. That single decision enforces three properties at once — the relationship is one to one, the credential disappears when the group is deleted, and no second vault can be created for the same group. The seat is an associative entity: conceptually it is the many-to-many link between a member and a group, but it carries a life cycle of its own (joined, expiring, leaving, refunded), so it is promoted to an entity with its own key. The monthly ledger is append-only and keyed on the pair (escrow, month), which makes double payment structurally impossible rather than merely unlikely.

The remaining design work is ordinary but consequential. A product's seat ceiling belongs to the product, not to the group, and is stored only once; copying it into every group would have created the transitive dependency that the third normal form exists to prevent. The escrow balance is the one deliberate exception: the remaining amount is derivable from the ledger, yet it is stored because it is read on every page while the ledger is written once a month. That exception is made safe by a CHECK constraint that makes the arithmetic a rule of the database, by a trigger that maintains the balance on every release, and by an audit query that compares the two and returns nothing when the system is healthy. The relations are otherwise in BCNF, which is strictly stronger than the third normal form the assignment requires.

Integrity is split deliberately between three layers. Keys, references and ranges live in the DDL, where they survive any client, including a careless INSERT typed into a database browser. Rules that depend on aggregates — the seat ceiling, the single default payment instrument, the automatic journal of credential changes, the balance identity — live in triggers, because they must hold for

every writer. Decisions that depend on who is acting — who may review a complaint, which state transition is legal — live in the application, because a database row cannot know which human is behind the connection.

4. The queries

Eighteen queries were written and executed against a seeded database of three hundred members, sixty groups, three hundred seats and more than two thousand monthly releases. They fall into three families. The first six are lookups a member performs: finding a group with one seat left on a particular service, listing the seats a person holds, reading the credential's last update. The next seven are the analysis a business runs: revenue by month, average time to fill a group, the services students actually occupy, the host leaderboard, refund rates, complaint handling time. The last five are controls that only make sense over time, and they are where the database earns its keep.

Two of them are worth describing in detail. The risk query asks for groups that changed their credential at least three times in thirty days and drew at least two complaints of the form "I cannot log in" in the same window. Neither condition is conclusive on its own — a host may legitimately reset a password, and a single complaint may be a mistake — but together they describe a pattern that deserves a human look. The query returns that queue; it does not punish anyone. The second is the consecutive-blocked-months query, which uses a window function to compare each month's payout status with the previous month's for the same escrow. A single blocked month is a dispute; two in a row is a group that has stopped paying without being closed, and that is a different problem requiring a different response.

The choice of these queries reflects a view about what a database is for. A schema that can only answer "what is in this row" is a filing cabinet. The interesting questions are relational and temporal: which groups are about to fill, which services are becoming expensive to support, which hosts are drifting from the norm, and how the platform's own revenue is moving month over month. Each of the eighteen queries answers one of those questions with a single statement that runs in milliseconds on commodity hardware.

5. What the data says

The seeded data is not decorative; it is the argument. Across three hundred seats the platform has taken HK$61,577 in prepayments, released HK$35,464 to hosts, and still holds HK$26,113 for months not yet delivered. The platform's own revenue — five per cent of every release — has grown from roughly HK$115 per month in the earliest full month to a peak of HK$176, which is a modest number and an honest one: at this scale the business is a proof of concept, not a profit centre.

The service ranking is more interesting than the revenue. Netflix in Hong Kong accounts for the largest share of student seats, followed by ChatGPT and YouTube. The ordering is not an accident of popularity; it tracks the size of the gap between the individual price and the family price. The platform grows where the discount is largest, which is exactly what the business model predicts and

what a spreadsheet of official prices would have suggested before a single user signed up. That agreement between model and data is the most persuasive evidence that the schema captures the business rather than merely storing it.

The risk analysis is deliberately unflattering. Five groups in the seeded data show the pattern of repeated credential changes and clustered access complaints, and their hosts' credit scores already reflect earlier penalties. The platform cannot prevent a determined host from lying; it can make the lie visible, freeze the money it is holding, and record who decided what. That is a weaker promise than "fraud is impossible", and a more credible one.

6. Limits and what comes next

The design has boundaries that are worth stating plainly. It models the escrow as a state machine rather than integrating a payment gateway, so no money actually moves; it stores the credential as encrypted text rather than implementing a secrets manager; it records that a host's bill was verified without performing that verification. It does not model partial months: a member who leaves on the twentieth day still receives that month's full twelfth, a simplification the business accepts. Comparisons across regions are made within a region, because no exchange rates are stored.

None of these is an oversight. Each is a decision about where the boundary of a database project should fall, and each has a natural next step. The market listing would benefit from a materialised view once the number of open groups grows beyond a few thousand. The ledger, which is append-only and grows by roughly one row per seat per month, should be partitioned by year once it approaches ten million rows. Closed groups older than two years should be archived, with their escrows settled and their credentials destroyed, because the platform should not hold a household passphrase longer than the relationship it supports.

What the project demonstrates, in the end, is that a small and unglamorous business problem — six strangers splitting a subscription — produces a data model of real subtlety. Money has to be held, released, frozen and returned; trust has to be measured and remembered; and every one of those movements has to be defensible a year later. Eleven relations, fifteen foreign keys and a handful of triggers are enough to do it, provided each one is placed for a reason.